Fundamentals of Artificial Intelligence - Lecture 6: Game Trees (26 Oct)

Lecture overview

Lecture 6 introduces game trees as a framework for adversarial search in multi-agent environments, specifically focusing on deterministic, zero-sum, two-player games like Chess and Checkers. The core mechanism discussed is the Minimax algorithm, which assumes rational agents where one player maximizes utility while the opponent minimizes it, recursively propagating values from terminal states up to the root to determine optimal moves. Because exhaustive search is computationally infeasible for complex games due to high branching factors and depth, the lecture emphasizes Alpha-Beta pruning as a critical optimization. This technique eliminates branches that cannot influence the final decision by maintaining bounds (α and β) on the best available options for MAX and MIN players, effectively doubling the solvable search depth with perfect move ordering while preserving the optimal minimax value.

To address resource limits in realistic scenarios where games cannot be searched to terminal leaves, the lecture covers bounded lookahead strategies combined with evaluation functions. These functions estimate the utility of non-terminal states using weighted linear sums of features or more complex nonlinear models, allowing agents to make decisions within a fixed depth horizon. The material highlights that deeper search generally yields superior play by revealing strategic consequences invisible at shallow horizons, even if the evaluation function is imperfect. Finally, the lecture extends these concepts to generalized minimax for multi-player, non-zero-sum games where utility is represented as tuples, and summarizes how these algorithms have driven advancements in AI across various domains, from solved games like Checkers to complex challenges in Go and video games.

Slide 1: Game Trees

This cover identifies Lecture 6 as Game Trees. The lecture develops adversarial search, pruning, and limited-depth evaluation.

Slide 2: Outline

The session covers game-search history, minimax for zero-sum games, alpha-beta pruning, and evaluation when computation cannot reach terminal states.

Slide 3: A Brief History of Game-Playing AI

The timeline shows game-playing programs as a source of AI ideas, with checkers, chess, Go, and Pacman as examples. It sets historical context rather than making game strength the only measure of AI.

Slide 4: Game Environment Properties

A game environment can be deterministic or stochastic, fully or partially observable, involve different numbers of players, alternate or simultaneous turns, and be zero-sum or general-sum. These properties determine how a strategy must account for an opponent, uncertainty, and information.

Slide 5: Standard Game Formulation

A standard game model specifies the initial state, whose turn it is, legal actions, the transition function, terminal test, and utility. A strategy maps states or histories to actions so the player can respond to different continuations.

Slide 6: Zero-Sum and General Games

In a zero-sum game one player's gain is the other's loss, so MAX and MIN can use one utility scale. General-sum games give players separate utilities and can include cooperation, indifference, or shifting competition.

Slide 7: Adversarial Search

Adversarial search plans while accounting for an opponent's possible replies. A search tree represents the contingent choices rather than a single fixed action sequence.

Slide 8: Single-Agent Trees

The single-agent tree propagates values from terminal outcomes toward the root. At a decision node the agent selects the child with the largest value; the next slide states this recurrence explicitly.

Slide 9: Value of a State

Terminal states have known utility. For a nonterminal single-agent state, V(s) is the maximum value among successor states, giving the best achievable outcome from that state.

Slide 10: Tic-Tac-Toe Game Tree

The tic-tac-toe tree expands legal moves into successor boards and eventual terminal outcomes. It visualizes how alternating choices create a tree whose leaves determine the values propagated upward.

Slide 11: Minimax Values

Minimax assigns MAX nodes the largest child value and MIN nodes the smallest child value. The leaf utilities are fixed; internal values represent the result of assuming each player chooses optimally.

Slide 12: Minimax Decision

Choose the action whose successor has the best minimax value. This protects MAX against an opponent that responds optimally, so the result is rational relative to the modeled game and utility.

Slide 13: Minimax Implementation

The recursive procedure returns utility at terminal states, maximizes over legal actions at MAX nodes, and minimizes at MIN nodes. The decision procedure chooses the root action with the best returned value.

Slide 14: Minimax Efficiency

With branching factor b and maximum depth m, exhaustive minimax takes O(b^m) time. Depth-first recursion needs O(bm) space. These growth rates make full-depth chess search infeasible at realistic branching and depth.

Slide 15: Game Tree Pruning

This section marker introduces pruning: avoid exploring branches that cannot affect the minimax choice while preserving the result.

Slide 16: Minimax Example

This worked tree propagates leaf utilities upward by taking maxima at MAX levels and minima at MIN levels. The example prepares for seeing which leaves alpha-beta can skip.

Slide 17: Alpha-Beta Example

Alpha is MAX's best value found so far along the path. Once a MIN node can do no better for MAX than that bound, its remaining children cannot change the root choice and can be pruned. Finding strong moves early improves pruning.

Slide 18: Alpha-Beta Quiz

This is an interactive poll prompt about alpha-beta search. It provides no fixed answer on the slide; solve the tree using the current alpha and beta bounds.

Slide 19: Alpha-Beta Quiz 2

This second poll continues the alpha-beta exercise. Treat it as a practice question and evaluate the branches under the shown move order.

Slide 20: Alpha-Beta Quiz Tree

The tree records partial values and alpha/beta bounds for the quiz. Update the bound after each examined child and stop a branch when it cannot affect an ancestor's choice; the slide leaves the exercise for the audience.

Slide 21: Alpha-Beta Pruning

At a MIN node, if the current value is at most alpha, MAX already has an alternative at least as good, so the remaining MIN children cannot change MAX's choice. Symmetrically, a MAX node can stop when its value is at least beta.

Slide 22: Alpha-Beta Implementation: MIN

The MIN procedure initializes v to positive infinity, updates it with each child's value, returns when v is at most alpha, and otherwise tightens beta. Alpha and beta carry the best bounds already available from MAX and MIN ancestors.

Slide 23: Alpha-Beta Implementation: MIN and MAX

The paired MIN and MAX procedures alternate recursive calls. MIN updates beta with its best current value and cuts off at $v \leq \text{alpha}$; MAX updates alpha and cuts off at $v \geq \text{beta}$.

Slide 24: Alpha-Beta Properties

Alpha-beta pruning preserves the root minimax value. Move ordering controls how much it prunes; iterative deepening can help find promising moves early. With ideal ordering, the time bound improves to about $O(b^{(m/2)})$, allowing roughly twice the depth for the same work.

Slide 25: Resource Limits

This section marker introduces limits on search resources and the practical need to estimate unfinished positions.

Slide 26: Bounded Lookahead and Evaluation

A search cannot usually reach terminal states in realistic games. Stop at a depth limit and apply an evaluation function to nonterminal positions; this enables a decision but removes the guarantee of perfect play.

Slide 27: Depth Matters

Deeper search usually improves decisions because it reveals more consequences, and it can compensate for a less precise evaluation function. The improvement is not guaranteed when the search horizon misses important events.

Slide 28: Pacman with Depth-2 Lookahead

The screenshot illustrates Pacman search with a two-ply lookahead. The agent evaluates a shallow set of possible continuations; the image does not show a measured score comparison.

Slide 29: Pacman with Depth-10 Lookahead

The same Pacman setup is shown with depth ten. A deeper search considers more future moves and costs much more computation; the screenshot alone does not identify a particular chosen move or performance gain.

Slide 30: Depth and Evaluation Trade-Off

Evaluation functions remain imperfect, so depth and feature quality trade off. More search can improve play, but computational cost grows quickly and a cutoff can still hide consequences just beyond the horizon.

Slide 31: Evaluation Functions

This section marker introduces evaluation functions, which assign approximate values to nonterminal states at the depth limit.

Slide 32: Evaluation Function Examples

A common evaluation function is a weighted sum of state features, $EVAL(s) = \Sigma i\ wi\ fi(s)$. In a board game, a feature might be the material difference. More complex functions can be learned, and quiescence search can extend analysis of unstable positions near the cutoff.

Slide 33: Evaluation for Pacman

The Pacman image gives a state for evaluation with ghosts and remaining dots. An evaluation function estimates how favorable such a nonterminal state is; the screenshot does not specify feature definitions or weights.

Slide 34: Generalized Minimax

For multiple players or non-zero-sum games, each terminal state can carry a utility tuple. Each player then chooses actions to maximize its own component, which can produce both cooperation and competition.

Slide 35: Summary

The lecture’s core points are minimax for adversarial choice, alpha-beta pruning as an exact way to reduce search, and depth-limited search with an evaluation function when full search is impossible. Efficient game search manages computation; it does not remove the need to define utility.

Key points to remember

Game search models alternating decisions with states, actions, transitions, terminal utilities, and a strategy. Minimax chooses optimally against an optimal opponent in a zero-sum game; alpha-beta pruning skips branches without changing the minimax result. Real games require a depth limit and an evaluation function, which estimates nonterminal states and removes the guarantee of perfect play.